

Ng-backmarking — an algorithm for constraint satisfaction

T Richards, Y Jiang and B Richards

Ng-backmarking with min-conflict repair, a ‘complete’ hybrid algorithm for solving constraint and optimisation problems, is presented. The search space of this combined approach is controlled by the ng-backmarking process, a method of learning constraints during search (at each failure point) that may be used to avoid the repeated traversing of failed paths in a search tree. In order to control the space of these learned constraints, or ‘no-goods’, it is important to assess what information can be learned on discovery of a failure by studying the constraints of the constraint-satisfaction problem. A strategy for maintaining the set of no-goods is introduced in order to support dynamic constraint satisfaction.

1. Introduction

Although many different search techniques have been shown to be successful for particular constraint-satisfaction problems (CSP) it seems that no single technique is best suited to solving all, of these techniques, it is possible to classify the underlying theory into four main categories — tree search, domain filtering, solution repair, and learning algorithms.

Tree search is the incremental extension of consistent labellings. On the discovery of a failure, tree search techniques backtrack and attempt to relabel variables. Backtracking may be chronological (backtracking to the most recently labelled variable), or intelligent (backjumping, backmarking, etc) [1]. Tree search follows the paths of the search space in a regular fashion by constructing and extending consistent incomplete labellings [2].

Domain-filtering techniques seek to remove values from the domains of variables before labelling. This requires consistency rules (rules for comparing the values in the domains). Consistency rules that have proved successful do not complete the constraint hypergraph. Examples include arc consistency and path consistency [3]. Domain filtering is performed together with tree search; a variable is labelled and then the subsequent or future variable domains are reduced [4]. There exist many real-world applications of these techniques within scheduling, planning and resource allocation, including warehouse location, cutting stock and bridge scheduling [5].

Solution repair or repair-based techniques work on (usually complete) labellings using some ‘relabeling#’ operation. Given an inconsistent labelling, solution repair

attempts to alter the labelling in some way to reduce inconsistencies. The min-conflict repair operator switches the value of a single variable involved in a violated constraint in order to reduce the overall number of conflicts [6]. It has been found that repair techniques are often suited to problems with a large solution space (high ratio of solutions to inconsistent labellings). For such problems they often find ‘good’ solutions in a short time, relative to other techniques. Repair-based techniques are naturally reactive (when a solution becomes inconsistent due to a change in the environment). There exist many real-world applications for which the solution space is large, including task scheduling for the Hubble space telescope [6].

Given a CSP and a search procedure, learning algorithms seek to build new constraints on the discovery of a failure or dead-end. (A failed labelling either violates some constraints or cannot be extended to a solution.) The learning is based on generating and recording labellings to subsets of the set of variables in a CSP that cannot be extended to a solution. These are called ‘no-goods’ [7]. This information is used to prune the subsequent search space. It has been shown that the technique of learning search knowledge from inconsistencies is equivalent to making some subset of the set of paths in the constraint hypergraph of a CSP path consistent [8].

This paper is concerned with presenting and understanding ng-backmarking, a learning algorithm with a repair-based method, called ‘min-conflict repair’ [9]. Ng-backmarking is then discussed and compared with learning algorithms combined with tree-search techniques. The motivation behind ng-backmarking is to allow the search to

follow local gradients in the search space whilst ensuring that a, preferably optimal, solution is found, if one exists. A method for extending ng-backmarking for dynamic constraint-satisfaction problems is then discussed.

2. Ng-backmarking with min-conflict repair

Ng-backmarking with min-conflict repair is a repair-based method that works on possibly inconsistent labellings. This algorithm builds, using the constraints of the CSP, a dynamically generated and simple set of no-goods. No-goods indicate labellings that cannot be extended to a solution and they are used to prune the search space.

- *CSP definition*

A CSP may be expressed by a set of variables $\{X_1, X_2, \dots, X_n\}$, each with an associated domain $\{D_1, D_2, \dots, D_n\}$, and a set of constraints $\{C_1, C_2, \dots, C_m\}$. Each C_i is a subset of $D_{i_1} \times D_{i_2} \times \dots \times D_{i_k}$, where the D_{ij} are the associated domains of the variables X_{ij} involved in the constraint C_i .

- *Definition of labelling in a CSP*

A labelling is a set of the form $P = \{(X_1, v_1), (X_2, v_2), \dots, (X_k, v_k)\}$, and indicates that X_i is assigned the value $v_i \in D_i$, where D_i is the domain of X_i , $1 \leq i \leq k$. A consistent labelling is a labelling that satisfies the constraints of the CSP. A complete labelling is a labelling of values to all variables in a CSP. A solution to a CSP is a complete and consistent labelling.

The learning aspect of ng-backmarking is now defined. This method of learning involves recording labellings to the variables of a CSP that cannot be extended to a solution. These recorded labellings are called no-goods and they are used in two ways. Firstly, the no-goods learned are used to direct the search and prune parts of the search space that contain no solutions. Secondly, the no-goods are used to ensure that this method will find a solution to a given CSP if one exists.

- *Definition of 'no-goods'*

For a given CSP, a no-good is a set of the form $ng = \{(X_1, v_1), \dots, (X_k, v_k)\}$. They indicate labellings that cannot be extended to a solution. No-goods are generated using the constraints of the CSP.

- *Example*

For a given CSP, if the labelling:

$$P = \{(X_1, v_1), (X_2, v_2), \dots, (X_k, v_k)\}$$

violates the constraint:

$$C_i(X_{i_1}, X_{i_2}, X_{i_3}, X_{i_4})$$

then a no-good is:

$$ng = \{(X_{i_1}, v_1), (X_{i_2}, v_2), (X_{i_3}, v_3), (X_{i_4}, v_4)\}$$

where $ng \subseteq P$

The variable and value-selection heuristics of ng-backmarking are based on the min-conflict heuristic [6]. Given the above notation it is possible to define this selection heuristic.

- *Min-conflict repair heuristic definition*

Given a CSP and a labelling $P = \{(X_1, v_1), (X_2, v_2), \dots, (X_k, v_k)\}$, the variable X_k is in conflict if $(X_k, v_k) \in P$ and it is involved in a violated constraint. The min-conflict strategy selects a variable in conflict and changes its value such that the resulting number of conflicts is minimised.

For any search procedure that uses no-goods to prune or restrict the search it is vital to the efficiency of such an algorithm to retain only minimal no-good sets. This is because the time and space complexity of such algorithms are directly proportional to the size of the set of no-goods.

- *Subsumption of definition*

A no-good ng subsumes a labelling (or no-good) P if $ng \subseteq P$. Given a set of no-goods, NG , it is stated a labelling P is subsumed by NG if there exists a no-good $ng \in NG$ such that $ng \subseteq P$. A set of no-goods NG is said to be minimal if for all $ng \in NG$ there does not exist $ng' \in NG$ such that $ng' \subset ng$.

The following notation shall be used in the next definitions:

— for a labelling, P , P^* is the set of variables labelled in P ,

— if P^* contains X , then $P-X$ is the labelling P with the assignment to X removed,

— the same notation applies to no-goods.

Given a set of no-goods it may be possible to derive other no-goods. In the following definition a set of no-goods are consistent if, and only if, they do not represent the assignment of different values to the same variable. For example, the set of no-goods $\{(X_1, v_1), (X_2, v_2), (X_3, v_3), (X_3, v_3)\}$ are consistent, whereas $\{(X_1, v_1), (X_2, v_2), (X_2, v_2), (X_3, v_3)\}$ are not consistent ($v_2 = v_2'$). It can be said that two no-goods ng_1 and ng_2 are consistent on all values except X if ng_1-X and ng_2-X are consistent.

- *Definition of merging*

For a given CSP, let NG be a set of no-goods, let X be a variable of the CSP, D_x the domain of X and P a labelling not subsumed by NG where $X \in P^*$. If, for all values $v_i \in D_x$, there exists a no-good $ng_i \in NG$ such that $ng_i \subseteq P(X, v_i)$, then another no-good is:

$$ng = \bigcup_i ng_i - X$$

- *Example*

For a given CSP, let $P = \{(Y, 1), (Z, 2)\}$, let X be a variable with $\{0, 1\}$ and suppose that there are the two no-goods $ng_1 = \{(X, 0), (Y, 1)\}$ and $ng_2 = \{(X, 1), (Z, 2)\}$. Then merging gives a third no-good:

$$\begin{aligned} ng &= \bigcup ng_i - X \\ &= \{(Y, 1)\} \cup \{(Z, 2)\} \\ &= \{(Y, 1), (Z, 2)\} \end{aligned}$$

Merging is fundamental for the efficiency of search algorithms that use no-goods to direct the search and prune the search space. The size of the set of no-goods and the choice of variable for repair depends greatly on merging.

- *NG-backmarking with min-conflict repair definition*

For a given CSP, P is a labelling, and NG is the set of no-goods (initially $NG = \emptyset$).

— 1 Consistency check

If P subsumed by NG then go to 3.

Else If P violates a constraint then go to 2.

Else P is consistent go to 4.

— 2 Generate no-good

Let C be a constraint violated by P . Let $\{X_{cl}, \dots, X_{cm}\}$ be the variables in C . Then $ng = \{(X_{cl}, v_{cl}), \dots, (X_{cm}, v_{cm})\}$ is a no-good. Remove all no-goods in NG subsumed by ng and add ng to NG . Go to 1.

— Repair

If $P \neq \emptyset$ then select a variable X in P^* that is involved in a no-good that subsumes P .

If select a value v in the domain of X such that the labelling resulting from flipping the value of X to v in P is not subsumed by NG and such that the number of conflicts is minimised. Go to 1.

Else find a set of no-goods $M \subseteq NG$ that prohibit all values for X . Merge M to produce a new no-good ng .

If $ng = \emptyset$ search has failed.

Else remove from NG any no-goods subsumed by ng .

Set P to $P - X$ (de-assign X). Add ng to NG and go to 1.

Else search has failed.

— 4 Labelling

If all variables are assigned then return P as a solution.

Else select an unassigned variable X . Select the value v in the domain of X such that the labelling resulting from adding the assignment (X, v) to P is not subsumed by NG and such that the number of conflicts is minimised. Go to 1.

3. Systematic and non-systematic search

Ng-backmarking with min-conflict repair combines a learning technique with a non-systematic search procedure, i.e. there is no predetermined variable ordering. The learning mechanism and the maintenance of the set of no-goods using subsumption checking and merging are independent of the search procedure. Learning techniques have been combined with many systematic search procedures where variables are assigned in a particular order. These are known as backtracking-with-learning algorithms. There follows a description of conflict-directed backjumping with jump-back learning [10], an example of systematic search with learning, using the above framework.

The systematic search used in the following is known as conflict-directed backjumping. Given a consistent labelling to some subset of the set of variables of a CSP, backjumping attempts to extend this to a complete labelling. On the discovery of a dead-end — where a variable cannot be consistently assigned a value — backjumping identifies a set of assigned variables that directly contribute to this dead-end and attempts to change the value of the most recently assigned variable in this set.

- *Conflict-directed backjumping definition*

For a given CSP, let P be a consistent labelling (initially $\emptyset$), and let K be an integer (initially 0).

— 1 Set parameters

Set K to $K+1$, let $JB = 0$ and go to 2.

— 2 Select

Select variable K and let D_k = domain of K . Go to 3.

— 3 Assign

If $D_k = \emptyset$ then set P to $P - \{\max(JB), \max(JB)+1, \dots, K-1\}$. Set K to $\max(JB)$ and set JB to $JB - \{\max(JB)\}$. Go to 2.

Else let veD_k

If $P \cup \{k, v\}$ is consistent, set P to $P \cup \{K, v\}$. Go to 1.

Else set D_k to $D_k - \{v\}$. Go to 4.

— 4 Jump-back set

Let C be a constraint violated by $P \cup \{(K, v)\}$. Let $V_c = \{X_{cl}, \dots, X_{cm}, K\}$ be the variables in C . Set JB to $JB \cup V_c - \{K\}$. Go to 3.

The jump-back set identified in step 4 may be used to build no-goods. This is known as jump-back learning. To append learning to the above algorithm requires a change to step 3 with an additional parameter, namely the set of no-goods. Initially $NG = 0$.

- *Jump-back learning definition*

Given a CSP, let P be a consistent labelling, K an integer and NG a set of no-goods.

— 3 Assign

If $D_k = 0$ then for $JB = \{X_{cl}, \dots, X_{cm}\}$,

$ng = \{X_{cl}, v_{cl}, \dots, (X_{cm}, v_{cm})\} \subseteq P$.

Set NG to $NG \cup \{ng\}$ and set P to $P - \{\max(JB), \max(JB)+1, \dots, K-1\}$.

Set K to $\max(JB)$ and set JB to $JB - \{\max(JB)\}$.

Go to 2.

Else let veD_k

If $P \cup \{(K, v)\}$ is consistent, set P to $P \cup \{(K, v)\}$. Go to 1.

Else set D_k to $D_k - \{v\}$. Go to 4.

There are many other ways of learning no-goods from dead-ends in systematic search. Other examples include value-based learning, graph-based learning, and deep-learning.

Before adding a no-good to the database in step 3 above, a subsumption checking phase may be used as in ng-backmarking. This will ensure that the minimal set of no-goods is retained.

- *Jump-back learning with subsumption checking definition*

Given a CSP, let P be a consistent labelling, k an integer and NG a set of no-goods.

— 3 Assign

If $D_k = 0$ then for $JB = \{X_{cl}, \dots, X_{cm}\}$, $ng = \{X_{cl}, v_{cl}, \dots, (X_{cm}, v_{cm})\} \subseteq P$. Remove from NG all no-goods subsumed by ng and set NG to $NG \cup \{ng\}$ and set P to $P - \{\max(JB), \max(JB)+1, \dots, K-1\}$. Set K to $\max(JB)$ and set JB to $JB - \{\max(JB)\}$. Go to 2.

Else let veD_k .

If $P \cup \{(K, v)\}$ is consistent set P to $P \cup \{(K, v)\}$. Go to 1.

Else set D_k to $D_k - \{v\}$. Go to 4.

4. Comparison of systematic and non-systematic search with learning

Figure 1 shows the constraint hypergraph for a binary CSP involving ten variables. Nodes represent variables with associated domains listed at the node, while arcs represent constraints between variables and are represented as sets of ordered pairs. For example, $C_{(Z,U)} = \{(a,a), (b,a), (b,b)\}$ represents a constraint between variables Z and U showing all inconsistent labellings to the variables.

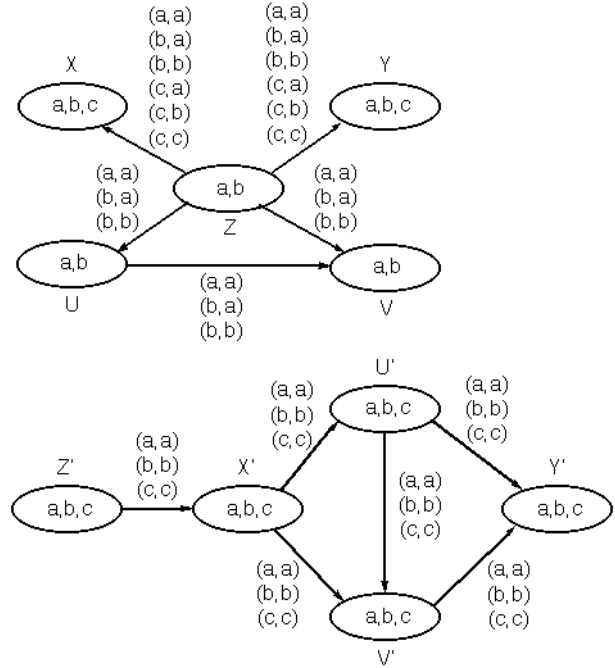


Fig 1 Constraint hypergraph for a binary CSP involving ten variables.

Table 1 shows an example of the search for a solution to this CSP using ng-backmarking. There are other possible ways for ng-backmarking to search for a solution. This is because the search procedure has an element of randomness both in the variable selection and value selection process.

The trace shows both merging and subsumption-checking working. After the initial inconsistent labelling has been made the no-good (1) (see Table 1) is generated from the violated constraint between X and Z . The value for X is then changed and no-good (2) is generated from another violated constraint. This process continues until no-good (8) is generated. At this point in the search Z is chosen for repair; however, there exist no values for Z such that the resultant labelling does not match a no-good. Z is then set as unassigned and the no-good (9) is generated by merging (5) and (8). The no-goods (5) and (8) are then removed using the subsumption-checking procedure and search continues. No-good (2) is generated by merging (1) and (11) which are removed at this point. Since U cannot now be assigned a

Table 1 Trace of ng-backmarking.

X	X'	Y	Y'	Z	Z'	U	U'	V	V'	no-goods added	removed
a	a	a	a	a	a	a	a	a	a	{(X,a),(Z,a)} (1)	
b	a	a	a	a	a	a	a	a	a	{(X',a),(Z',a)} (2)	
b	a	a	a	a	b	a	a	a	a	{(Y,a),(Z,a)} (3)	b
b	a	b	a	a	b	a	a	a	a	{(U',a),(V',a)} (4)	
b	a	b	a	a	b	a	a	a	c	{(Z,a),(U,a)} (5)	
b	a	b	a	b	b	a	a	a	c	{(X,b),(Z,b)} (6)	
c	a	b	a	b	b	a	a	a	c	{(Y,b),(Z,b)} (7)	
c	a	c	a		b	a	a	a	c	{(U,a)} (9)	(5), (8)
c	a	c	a		b	b	a	a	c	{(U,b),(V,a)} (10)	
c	a	c	a		b	b	a	b	c	{(U,b),(V,b)} (11)	
c	a	c	a		b	b	a		c	{(U,b)} (12)	(10,), (11)
c	a	c	a		b		a		c	{ } (13)	

Table 2 Trace of conflict-directed backtracking with jump-back learning.

X	X'	Y	Y'	Z	Z'	U	U'	V	V'	no-goods added	removed
a											
a	a										
a	a	a									
a	a	a	a								
a	a	a	a							{(X,a)} (1)	
b											
b	a										
b	a	a									
b	a	a	a								
b	a	a	a							{(Y,a)} (2)	
b	a	b									
b	a	b	a								
b	a	b	a	a							
b	a	b	a	a	b						
b	a	b	a	a	b	b					
b	a	b	a	a	b	b	b				
b	a	b	a	a	b	b	b			{(U,b)} (3)	
b	a	b	a	a	b					{(Z,a)} (4)	
b	a	b								{(X,b)} (5)	
c											
c	a										
c	a	b									
c	a	b	a								
c	a	b	a							{(X,c),(Y,b)} (6)	
c	a	c									
c	a	c	a								
c	a	c	a	b							
c	a	c	a	b	b						
c	a	c	a	b	b					{(Z,b)} (7)	
c	a	c	a							{(X,c)} (8)	(6)
										{ } (9)	

value such that the resultant labelling does not match a no-good, the process merges (9) and (12) which generates the no-good (13) (failure) and the search terminates.

Table 2 shows the trace of conflict-directed backtracking with jump-back learning applied to the above problem. The algorithm generates no-goods using the conflict set and decides to which variable it will backtrack. There are no removed no-goods, although there exist cases in which this operation may be of great importance to efficiency.

The trace shows the learning procedure for jump-back learning working for a specific problem. It also shows subsumption checking in operation.

Two questions may now be addressed to some extent using the example: ‘What is gained by using learning’, and: ‘What are the differences between these two search procedures?’

To answer the first question it is worth noting that for any CSP it can be shown that conflict-directed backjumping outperforms chronological backtracking. With the addition of learning, much of the search done by conflict-directed backjumping is cut out. For example, when the value of X is changed to c , then, without the learning procedure, the search would try the value a for Y . No-good (2) avoids this redundant search and forces Y to be b . The same happens when the value of Y is changed from b to c . The choice of value for Z must be b since no-good (4) prohibits the value a .

Ng-backmarking will find a, preferably optimal, solution if one exists, whereas most non-systematic search methods do not guarantee this. The gain in efficiency over systematic search techniques lies in the capacity for ng-backmarking to follow local gradients in the search space. However, even with the merging and subsumption-checking procedures, there is a possibility that the no-good set may become prohibitively large. Heuristics for variable selection must therefore be chosen to avoid such a situation.

The main difference between the above two methods is the predetermined ordering of variables in systematic search as opposed to the freedom of choice in ng-backmarking. The problem of Fig 1 is a composite problem made up of two disjoint sub-problems. Ng-backmarking discovers that there are no solutions to this composite problem by attempting to solve each one of the sub-problems separately. This means that the work done by ng-backmarking to solve a composite problem is a function of the sum of the work done in solving each sub-problem separately. Since the systematic search may backtrack over variables in separate sub-problems, this means that sub-problems may have to be solved more than once. Thus the work done by any systematic search procedure to solve a

composite problem is a function of the product of the work done in solving each sub-problem separately.

Many real-world applications are made up of disjoint or loosely connected sub-problems. Ng-backmarking, it seems, is well suited to exploiting the structure of such applications without requiring mathematical analysis.

5. Dynamic support of no-goods

In a dynamic CSP, constraints may be added, deleted or modified.

• Dynamic CSP definition

A dynamic CSP is a set of CSPs $A_1, A_2, A_3, \dots, A_n$, such that A_i is the result of a change in A_{i-1} , either a restriction (constraint addition), a relaxation (constant removal) or some combination of the two.

Clearly, if constraints are added then any no-goods learned prior to this remain no-good. However, if constraints are deleted then certain no-goods may have to be removed to maintain the consistency of the no-good set with respect to the new CSP. To deal with such a dynamic environment, the supporting constraints are attached to the no-goods [11]. A new notation is required to represent no-goods in the dynamic environment.

• Definition of no-goods for dynamic CSPs

Given a dynamic CSP no-goods represent labellings to variables that cannot be extended to a solution. They involve a constraint set which support the no-good. If any constraint in the constraint set is removed at any stage in the dynamic CSP then the no-good is no longer supported and is removed from the no-good database. A no-good is written in the form

$$\langle \{(X,a),(Z,a)\}, \{C(Z,X)\} \rangle$$

If the constraint $C(Z,X)$ is subsequently removed then this no-good is removed from the database.

Subsumption checking and merging now involve checking the supporting constraints for no-goods.

- *Definition of subsumption checking for dynamic CSPs*

Suppose that a no-good labelling $\langle ng, c \rangle$ is generated, where c is the set of supporting constraints; then, before adding this to the set of no-goods, NG , already learned:

- find all $\langle ng_i, c_i \rangle \in NG$ such that $ng \supseteq ng_i$,
- for all such no-goods, if $c \subseteq c_i$ then remove $\langle ng_i, c_i \rangle$ from NG .

- *Definition of merging for dynamic CSPs*

For a given CSP, let NG be a set of no-goods, let X be a variable of the CSP, D_x the domain of X , and P a labelling not subsumed by NG with $X \in P^*$. If, for all values $v_i \in D_x$, there exists a no-good $\langle ng_i, c_i \rangle \in NG$ such that $ng_i \{ (X, v_i) \}$, then another no-good is:

$$ng = \langle \bigcup_i ng_i - X, \bigcup_i c_i \rangle$$

- *Example*

For a given CSP let the domain of X be $\{0,1\}$ and suppose that there are the two no-goods $ng_1 = \langle \{(X,0), (Y,1)\}, \{C(X,Y)\} \rangle$ and $ng_2 = \langle \{(X,1), (Z,2)\}, \{C(X,Z)\} \rangle$. Then merging gives a third no-good:

$$\begin{aligned} ng_3 &= \langle \bigcup_{1 \leq i \leq 2} ng_i - X, \bigcup_{1 \leq i \leq 2} c_i \rangle \\ &= \langle \{(Y,1), (Z,2)\}, \{C(X,Y), C(X,Z)\} \rangle \end{aligned}$$

6. Conclusions

A similar technique to ng-backmarking has been developed, called partial-order dynamic backtracking [12]. The motivations for this algorithm are the same as for ng-backmarking, namely, allowing the search to follow local gradients in the search space while retaining the guarantee of a solution if one exists. Control of the search is done using no-goods and these are used to avoid the repeated search of redundant parts of the search space. Partial-order dynamic backtracking, however, differs from ng-backmarking in that information in the form of no-goods is removed during search. This is an attempt to avoid the set of no-goods becoming prohibitively large. A set of safety conditions or a partial ordering on the variables is generated dynamically during search and is used to assess when a no-good may be removed. Removing no-goods using an operation involving such safety conditions involves extra computation. A comparison of ng-backmarking with partial-order dynamic backtracking on specific applications is necessary to fully understand the differences and assess the merits of the two approaches.

This paper has presented an algorithm for solving constraint satisfaction problems. The type of problems best suited to this method of search have been suggested, although much testing on specific applications is now necessary. It has been shown where ng-backmarking could

provide advantages over systematic search. Since the algorithm also guarantees to find a solution to a CSP, if one exists, it is an improvement on most non-systematic search procedures. The scalability of ng-backmarking must be investigated. As mentioned in section 4, the heuristics for variable selection have significant impact on the efficiency and scalability of ng-backmarking. The heuristic for variable selection in this paper is based on choosing a variable involved in a no-good. It remains possible to select amongst these variables using further heuristics. This forms part of the future research on specific applications.

The framework in which the various learning algorithms have been discussed in this paper provides a means of comparing different learning techniques. Future research includes looking at domain-filtering techniques compared to these learning techniques (as well as together with them). Research into the kinds of problems best suited to these algorithms is possible.

Repair-based techniques are often well suited to dynamic CSPs, since they may be used to repair solutions that have become inconsistent due to a change in the environment. The no-goods of ng-backmarking may be used to prune the search for a solution in the dynamic environment and updating the set of no-goods is a simple set operation. How ng-backmarking performs for different dynamic CSPs forms a further area of research.

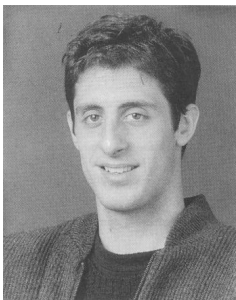
Acknowledgements

The authors would like to thank N Azarmi, J Lever, H El-Sakkout, A El-Kholy, and all members of IC-Parc for helpful comments and discussions.

References

- 1 Prosser P: 'Hybrid algorithms for the constraint satisfaction problem', *Computational Intelligence*, 9 (1993).
- 2 Freuder E and Wallace R: 'Partial constraint satisfaction', *Artificial Intelligence*, 52 (1992).
- 3 Mackworth A: 'Consistency in networks of relations', *Artificial Intelligence*, 8 (1989).
- 4 Nadel B: 'Constraint satisfaction algorithms' *Computational Intelligence*, 8, No 4 (1989).
- 5 Van Hentenryck P: 'Constraint satisfaction in logic programming', MIT Press (1989).
- 6 Minton S, Johnson M, Philips A and Laird P: 'Minimising conflicts: a heuristic repair method for constraint satisfaction and scheduling problems', *Artificial Intelligence*, 58 (1992).

- 7 Stallman R M and Sussman G J: 'Forward reasoning and dependency directed backtracking in a system for computer aided circuit analysis', *Artificial Intelligence*, 9 (1977).
- 8 Dechter R: 'Enhancement schemes for constraint processing: back-jumping, learning and cutset decomposition', *Artificial Intelligence*, 41 (1989).
- 9 Jiang Y, Richards T and Richards B: 'No-good backmarking with min-conflict repair in constraint satisfaction and optimisation', in *Proc of 2nd International Workshop in PPCP-94*, Washington USA (1994).
- 10 Dechter R: 'Dead-end driven learning', *Proceedings AAAI-94* (1994).
- 11 Schiex T and Verfaillie G: 'No-good recording for static and dynamic constraint satisfaction problems', *Proceedings of the 5th IEEE International Conference on Tools with Artificial Intelligence* (1993).
- 12 Ginsberg M: 'GSAT and dynamic backtracking', *Principles and practices in Constraint Programming* (1994).



Tom Richards received a BSc in mathematics in 1991 from University of London and his MSc in mathematics in 1992 also from University of London.

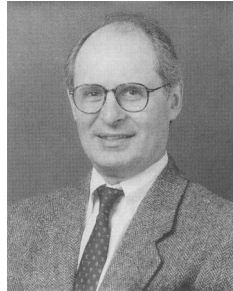
He has been studying for a BT sponsored PhD degree at IC-Parc (Imperial College).

His research work covers techniques for constraints satisfaction and optimisation problems.



Yuejun Jiang received both a BSc degree in 1983 and a PhD degree in 1986 in Computer Science from University of Manchester. Since then he has held an ICL fellowship at Essex University, a BT fellowship at Cambridge and an Advance-EP SRC fellowship at IC-Parc, Imperial College.

His main research interests include logical foundation of AI and techniques for constraint satisfaction and optimisation.



Professor Barry Richards is Director of the Centre for Planning and Resource Control (IC-Parc) at Imperial College, where he also holds an established Chair of Computing Science.

He received a BA degree from the University of Notre Dame (Indiana) in 1964, and a PhD from the University of Cambridge in 1972. He joined the Department of Computing at Imperial College in May 1989, having previously been Professor of Cognitive Science in the University of Edinburgh and Director of the Centre for Cognitive Science.

His current research interests are in planning and scheduling, temporal reasoning, and constraint solving, covering both their theoretical foundations and practical implementations.